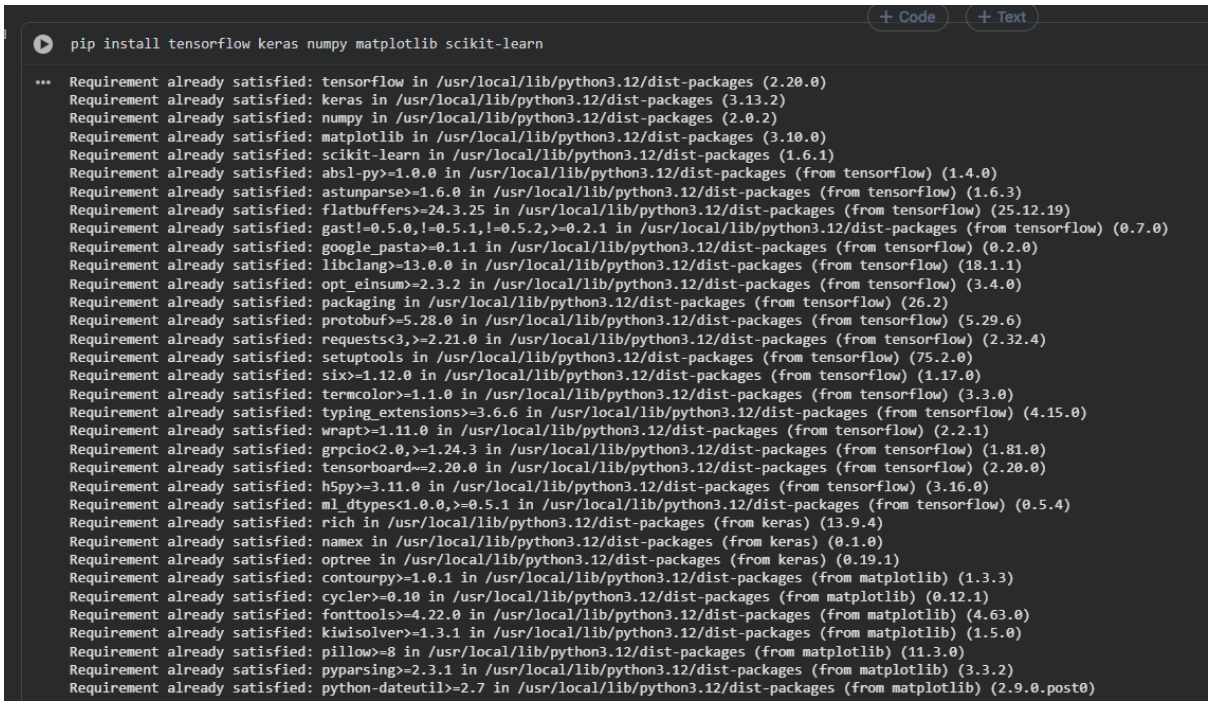
Deep Learning Fundamentals + Neural Networks

Task 1: Deep Learning Environment Setup

Set up a Python environment for deep learning development using TensorFlow and Keras.



Task 2: Research Neural Network Concepts

Neural Network

Definition

A Neural Network is a computational model inspired by the human brain. It consists of interconnected nodes called neurons that process information and learn patterns from data.

The main goal of a neural network is to recognize relationships in data and make predictions or classifications.

Real-World Examples

1. Face Recognition System
2. Voice Assistants
3. Self-Driving Cars
4. Recommendation Systems
5. Medical Diagnosis
6. Fraud Detection

7. Language Translation

Layers in Neural Networks

Input Layer

The input layer receives data from external sources.

Example:

For Iris Dataset:

- Sepal Length
- Sepal Width
- Petal Length
- Petal Width

The input layer contains 4 neurons.

Hidden Layer

Hidden layers perform computations and extract patterns.

Functions:

- Feature Extraction
- Pattern Recognition
- Non-linear Learning

Deep neural networks contain multiple hidden layers.

Output Layer

Produces final prediction.

Examples:

Binary Classification:

- 0 = No
- 1 = Yes

Multi-Class Classification:

- Setosa
- Versicolor
- Virginica

Activation Functions

ReLU (Rectified Linear Unit)

Characteristics:

- Fast computation
- Solves vanishing gradient problem
- Most commonly used

Used In:

- Hidden Layers
- CNNs
- Deep Neural Networks

Advantages:

- Efficient
- Simple
- Faster convergence

Sigmoid

Output Range:

0 to 1

Used In:

- Binary Classification
- Output Layers

Advantages:

- Probability interpretation

Disadvantages:

- Vanishing gradient problem

Tanh

Output Range:

-1 to 1

Used In:

- Hidden Layers
- Recurrent Neural Networks

Advantages:

- Zero-centered outputs

Disadvantages:

- Computationally expensive

Softmax

Output:

Probability distribution.

Used In:

- Multi-class classification
- Final output layer

Example:

Digit Recognition:

Digit 0 : 0.01
Digit 1 : 0.02
Digit 2 : 0.90
Digit 3 : 0.01
...

Prediction = Digit 2

Training Concepts

Epoch

One complete pass through the training dataset.

Example:

Dataset Size = 10,000 images

Epoch = 1

Model sees all 10,000 images once.

Batch Size

Number of samples processed before updating weights.

Example:

Dataset = 10,000

Batch Size = 100

Updates per Epoch:

10000 / 100 = 100 updates

Learning Rate

Controls how much weights change during training.

Small Learning Rate:

- Slow training
- Better accuracy

Large Learning Rate:

- Fast training
- May overshoot optimum solution

Common Values:

0.1
0.01
0.001

Loss Function

Measures prediction error.

Examples:

Binary Crossentropy

Binary Classification

Categorical Crossentropy

Multi-class Classification

Mean Squared Error

Regression Problems

Goal:

Minimize loss.

Optimizer

Updates model weights.

Popular Optimizers:

1. SGD
2. Adam
3. RMSProp
4. Adagrad

Adam is most commonly used because it combines speed and stability.

Gradient Descent

Optimization algorithm used to reduce loss.

Steps:

1. Predict output
2. Calculate loss
3. Compute gradients
4. Update weights
5. Repeat

The neural network gradually learns by reducing errors during training.

Importance of Deep Learning

Deep learning can automatically learn complex features from raw data without manual feature engineering.

Applications include:

- Computer Vision
- Natural Language Processing
- Healthcare
- Finance

Task 3: Build Your First Neural Network

Objective:

Build a classification model using the Iris Dataset.

Task 3: Build Your First Neural Network

```
import numpy as np
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.utils import to_categorical

iris = load_iris()
x = iris.data
y = iris.target

y = to_categorical(y)
x_test, x_train, y_test, y_train = train_test_split(x, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
x_train = scaler.fit_transform(x_train)
x_test = scaler.transform(x_test)

model = Sequential()
model.add(Dense(10, activation='relu', input_shape=(4,)))
model.add(Dense(10, activation='relu'))
model.add(Dense(3, activation='softmax'))

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
model.fit(x_train, y_train, epochs=125, batch_size=8, verbose=1)

loss, accuracy = model.evaluate(x_test, y_test)

predictions = model.predict(x_test)

print("\nAccuracy:",accuracy *100,"%")
print("\nSample Predictions:")
print(np.argmax(predictions[:5], axis=1))
```

```
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9667 - loss: 0.1819
Epoch 119/125
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9667 - loss: 0.1796
Epoch 120/125
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9667 - loss: 0.1770
Epoch 121/125
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9667 - loss: 0.1749
Epoch 122/125
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9667 - loss: 0.1725
Epoch 123/125
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.9667 - loss: 0.1701
Epoch 124/125
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.9667 - loss: 0.1677
Epoch 125/125
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9667 - loss: 0.1659
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.9333 - loss: 0.2495
WARNING:tensorflow:5 out of the last 9 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_step_on_data_distributed at 0x7d8d549b
1/4 ━━━━━━━━━━━━━━━━━━━━ 0s 48ms/stepWARNING:tensorflow:6 out of the last 12 calls to <function TensorFlowTrainer.make_predict_function.<locals>.one_
4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step

Accuracy: 93.33333373069763 %

Sample Predictions:
[0 0 1 0 0]
```

Task 4: Experiment With Network Architecture

```python
import time
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.utils import to_categorical

iris = load_iris()
x = iris.data
y = to_categorical(iris.target)

x_train, x_test, y_train, y_test = train_test_split(x,y , test_size=0.2, random_state=42)

scaler = StandardScaler()
x_train = scaler.fit_transform(x_train)
x_test = scaler.transform(x_test)

experiments = [
    { "name": "model 1","layers":[8],"activation":"relu", "epochs":50},
    { "name": "model 2","layers":[16,8],"activation":"relu", "epochs":100},
    { "name": "model 3","layers":[32,16,8],"activation":"tanh", "epochs":100},
]

for exp in experiments:
  model = Sequential()
  model.add(Dense(exp["layers"][0], activation=exp["activation"], input_shape=(4,)))
  for neurons in exp["layers"][1:]:
    model.add(Dense(neurons, activation=exp["activation"]))
  model.add(Dense(3, activation='softmax'))

  model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

  start = time.time()
  model.fit(x_train, y_train, epochs=exp["epochs"], batch_size=8, verbose=0)
```

```python
  training_time = time.time() - start
  loss, accuracy = model.evaluate(x_test, y_test, verbose=0)

  print(f'\n{exp['name']}')
  print(f'Training Time: {training_time:.2f} seconds')
  print(f'Accuracy: {accuracy * 100:.2f}%')
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)

model 1
Training Time: 5.93 seconds
Accuracy: 96.67%

model 2
Training Time: 7.93 seconds
Accuracy: 100.00%

model 3
Training Time: 8.86 seconds
Accuracy: 100.00%
```

Explain why some models perform better.

Some neural network models perform better because they learn patterns from data more effectively.

1. More Hidden Layers

- More hidden layers help the model understand complex patterns.
- Too few layers may not learn enough information.

2. More Neurons

- More neurons give the model greater learning power.
- Too many neurons can make the model memorize data instead of learning.

3. Better Activation Functions

- ReLU usually works better because it trains faster.
- Sigmoid and Tanh can be slower in deep networks.

4. More Training Epochs

- More epochs allow the model to learn better.
- Too many epochs may cause overfitting.

5. Proper Learning Rate

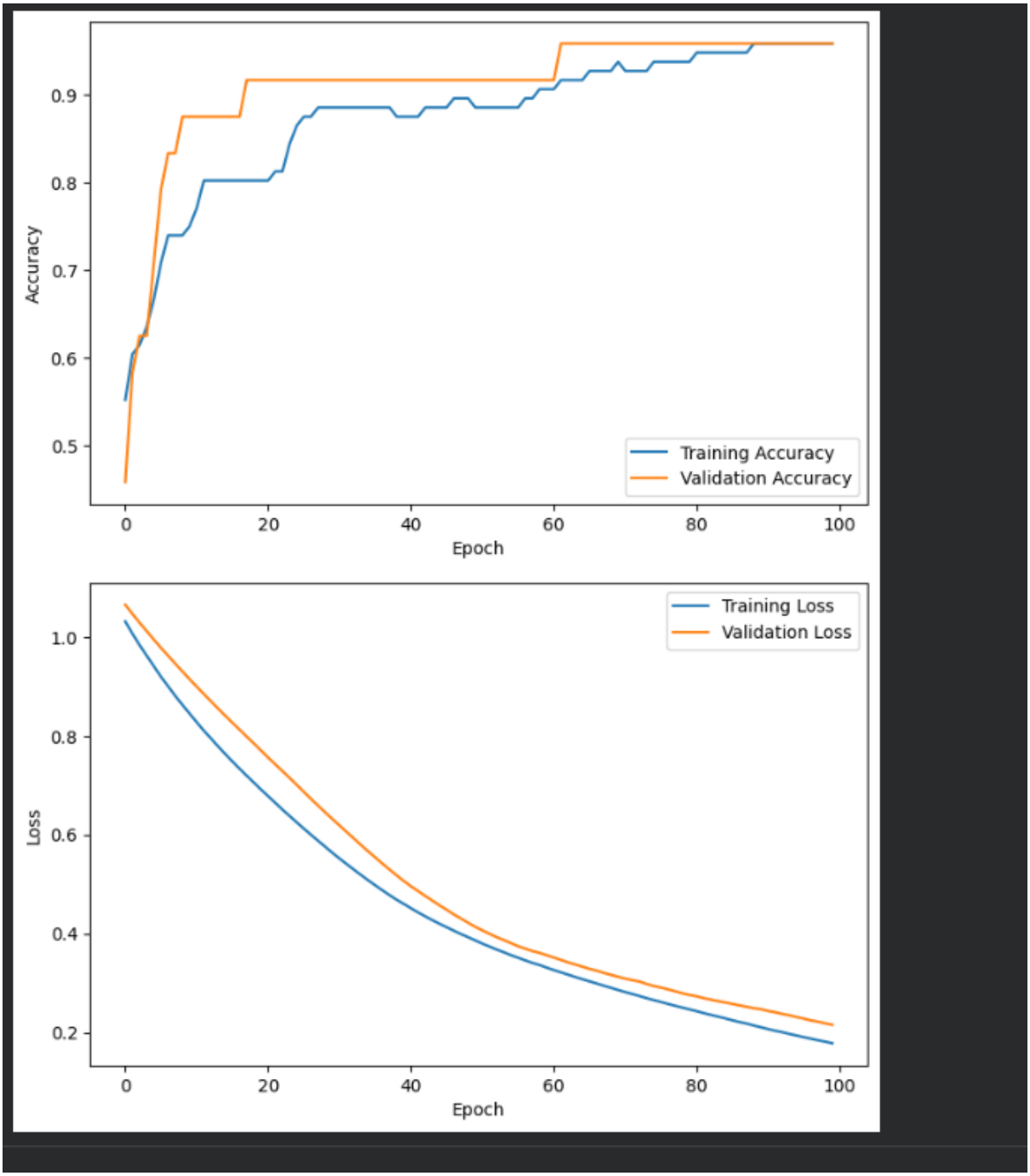
- A good learning rate helps the model learn efficiently.
- If it is too high, the model may miss the correct solution.
- If it is too low, training becomes very slow.

6. Normalized Data

- Scaling data to a similar range helps the model train faster and achieve better accuracy.

Task 5: Understand Loss and Accuracy

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.utils import to_categorical

iris = load_iris()
x = iris.data
y= to_categorical(iris.target)

x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)

scaler = StandardScaler()
x_train = scaler.fit_transform(x_train)
x_test = scaler.transform(x_test)

model = Sequential([Dense(16, activation='relu', input_shape=(4,)),
                    Dense(16, activation='relu'),
                    Dense(3, activation='softmax')

])

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

history = model.fit(x_train,y_train, epochs=100, validation_split=0.2, verbose=1)


plt.figure(figsize=(8,5))
plt.plot(history.history['accuracy'], label='Training Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.savefig('accuary_graph.png')
plt.show()

plt.figure(figsize=(8,5))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.savefig('loss_graph.png')
plt.show()
```



Task 6: Build a Digit Recognition Model

MNIST Dataset

Implement:

- Load MNIST data
- Normalize images
- Build Neural Network
- Train model
- Evaluate accuracy

- Predict sample images

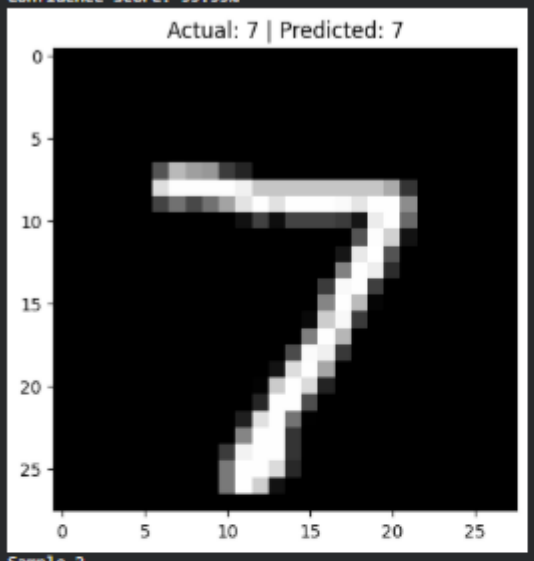
Display:

- Actual digit
- Predicted digit
- Confidence score

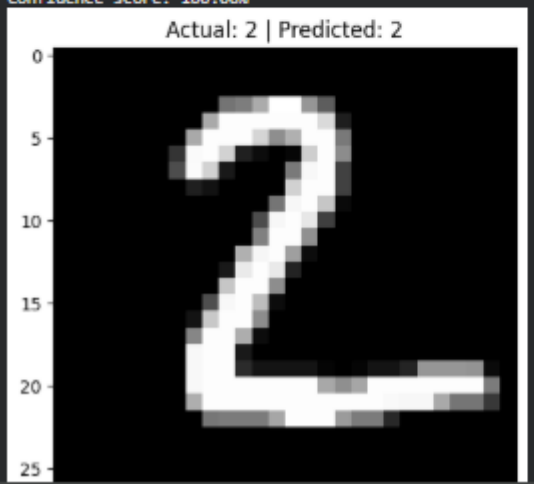
Task 6: Build a Digit Recognition Model

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.utils import to_categorical

(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train = x_train / 255.0
x_test = x_test / 255.0

y_train_cat = to_categorical(y_train, num_classes=10)
y_test_cat = to_categorical(y_test, num_classes=10)

model = Sequential([
     Flatten(input_shape=(28,28)),
     Dense(128, activation='relu'),
     Dense(64, activation='relu'),
     Dense(10, activation='softmax')
 ])

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
model.fit(x_train, y_train_cat, epochs=5, batch_size=32, validation_split=0.1)
loss, accuracy = model.evaluate(x_test, y_test_cat)
print(f'Test Accuracy: {accuracy * 100:.2f}%')

predictions = model.predict(x_test)
for i in range(5):
 predicted_digit = np.argmax(predictions[i])
 confidence = np.max(predictions[i]) * 100

 print(f"Sample {i+1}")
 print(f"Actual digit : {y_test[i]}")
 print(f"Predicted Digit: {predicted_digit}")
 print(f"Confidence score: {confidence:.2f}%")

 plt.imshow(x_test[i], cmap='gray')
 plt.title(f"Actual: {y_test[i]} | Predicted: {predicted_digit}")
 plt.show()
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step
Sample 1
Actual digit : 7
Predicted Digit: 7
Confidence score: 99.99%
```




```
Sample 2
Actual digit : 2
Predicted Digit: 2
Confidence score: 100.00%
```



Task 7: Model Saving and Loading

```python
import numpy as np
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import Sequential, load_model
from tensorflow.keras.layers import Dense, Flatten
from tensorflow.keras.utils import to_categorical

(x_train, y_train), (x_test, y_test) = mnist.load_data()
x_train = x_train / 255.0
x_test = x_test / 255.0

y_train = to_categorical(y_train, 10)
y_test = to_categorical(y_test, 10)

model = Sequential([
     Flatten(input_shape=(28,28)),
     Dense(128, activation='relu'),
     Dense(64, activation='relu'),
     Dense(10, activation='softmax')
 ])

model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
model.fit(x_train, y_train_cat, epochs=5, batch_size=32, validation_split=0.1)

model.save('digit_model.h5')
print('Model saved as digit_model.h5')

loaded_model = load_model('digit_model.h5')
print("Model loaded successfully")

loss, accuracy = loaded_model.evaluate(x_test, y_test)
print(f'Test Accuracy: {accuracy * 100:.2f}%')

prediction = loaded_model.predict(x_test[:1])

predicted_digit = np.argmax(prediction)
confidence = np.max(prediction) * 100
```

```python
print(f"Predicted Digit: {predicted_digit}")
print(f"Confidence Score: {confidence:.2f}%")

print("\n Prediction Result")
print("Predicted Digit:", predicted_digit)
print(f"Confidence Score: {confidence:.2f}%")
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models
  super().__init__(**kwargs)
Epoch 1/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 9s 5ms/step - accuracy: 0.9256 - loss: 0.2554 - val_accuracy: 0.9695 - val_loss: 0.1042
Epoch 2/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.9674 - loss: 0.1074 - val_accuracy: 0.9713 - val_loss: 0.0977
Epoch 3/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.9772 - loss: 0.0739 - val_accuracy: 0.9692 - val_loss: 0.1078
Epoch 4/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.9823 - loss: 0.0571 - val_accuracy: 0.9765 - val_loss: 0.0825
Epoch 5/5
1688/1688 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.9855 - loss: 0.0454 - val_accuracy: 0.9782 - val_loss: 0.0800
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We recommend using instead the nati
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile_metrics` will be empty until you train or evaluate the model.
Model saved as digit_model.h5
Model loaded successfully
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.9779 - loss: 0.0821
Test Accuracy: 97.79%
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 77ms/step
Predicted Digit: 7
Confidence Score: 100.00%

 Prediction Result
Predicted Digit: 7
Confidence Score: 100.00%
```

Task 8: AI Engineering Analysis Report

1.Why Deep Learning Models Require Large Datasets

Deep learning models contain thousands, millions, or even billions of parameters. These parameters must learn meaningful patterns from data. Large datasets provide diverse examples that help models generalize effectively. Small datasets often lead to overfitting, where the model memorizes training samples rather than learning underlying patterns.

For example, a digit recognition system trained on only 100 images may perform poorly on new handwriting styles. However, training on tens of thousands of examples allows the network to recognize different writing patterns and improve accuracy.

Large datasets also reduce bias and improve robustness. They help models learn variations in lighting, angles, noise, language usage, accents, and other real-world conditions.

2.What is the role of activation functions?

Activation functions introduce non-linearity into neural networks. Without them, a neural network would behave like a simple linear model regardless of the number of layers.

Activation functions allow networks to learn complex relationships and patterns.

Examples:

- ReLU enables efficient deep learning.
- Sigmoid produces probabilities.
- Tanh provides balanced outputs.
- Softmax handles multi-class predictions.

They are essential for enabling neural networks to solve real-world problems such as image recognition and language processing.

3.Why is data normalization important?

Normalization scales input values into a common range.

Benefits:

1. Faster training
2. Improved convergence

3. Stable gradients
4. Better accuracy

For image data, dividing pixel values by 255 scales values between 0 and 1.

Without normalization, large feature values may dominate learning and slow optimization.

4.How does a neural network learn?

Neural networks learn through an iterative process.

1. Receive input data.
2. Generate predictions.
3. Compute prediction error using a loss function.
4. Use backpropagation to calculate gradients.
5. Update weights using an optimizer.
6. Repeat across multiple epochs.

Over time, prediction errors decrease and accuracy improves.

Learning is essentially a process of continuously adjusting internal parameters to minimize loss.

5.How is deep learning connected to modern LLMs?

Large Language Models (LLMs) are advanced deep learning systems built using transformer architectures. Models such as GPT, Gemini, and other modern AI systems use deep neural networks with billions of parameters.

LLMs learn language patterns by training on enormous text datasets. They utilize deep learning techniques including:

- Neural Networks
- Backpropagation
- Gradient Descent
- Attention Mechanisms
- Optimization Algorithms

Deep learning forms the foundation of modern artificial intelligence. Without advances in neural network architectures, large datasets, and powerful computing resources, today's LLMs would not exist.

As computational power and data availability continue to increase, deep learning will remain a driving force behind innovations in natural language processing, computer vision, robotics, healthcare, and scientific research.

Bonus Challenge

Features:

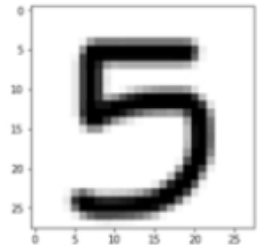
1. Load trained model
2. Upload digit image
3. Preprocess image
4. Predict digit
5. Show confidence score

```python
import streamlit as st
import numpy as np
from PIL import Image
from tensorflow.keras.model import load_model

model = load_model('digit_model.h5')
st.title("Handwritten Digit Prediction App")

uploaded_file = st.file_uploader("Upload an digit image ", type=["jpg", "jpeg", "png"])

if uploaded_file is not None:
  image = Image.open(uploaded_file).convert('L')
  st.image(image, caption='Uploaded Image', width=200)

  image = image.resize((28,28))
  image_array = np.array(image)
  image_array = 255 - image_array
  image_array = image_array / 255.0
  image_array = image_array.reshape(1, 28,28)

  prediction = model.predict(image_array)
  predicted_digit = np.argmax(prediction)
  confidence = np.max(prediction) * 100

  st.success(f"Predicted Digit: {predicted_digit}")
  st.info(f"Confidence Score: {confidence:.2f}%")
```

Handwritten Digit Prediction App

Upload an digit image

download.png
2.5KB



Uploaded Image

Predicted Digit: 1

Confidence Score: 19.29%